Модели регрессии

Цель работы

Познакомиться с основными приемами работы с моделями регрессии в scikit-learn.

Задания для выполнения

1. Загрузите встроенные датасет о ценах на недвижимость в Калифорнии.
2. Постройте модель регрессии для предсказания цены конкретного объекта.
3. Оцените качество построенной модели с помощью визуализации и коэффициента детерминации.
4. Постройте альтернативную полиномиальную модель, сравните ее с предыдущей.

Методические указания

В качестве исходных данных будем использовать датасет о ценах на объекты недвижимости в Калифорнии. Это один из известных обучающих наборов данных. Он встроен в библиотеку *sklearn*, так что его не нужно загружать или скачивать отдельно. Для начала работы импортируем стандартные необходимые библиотеки:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Сперва загрузим исходный набор данных:

```
from sklearn.datasets import fetch_california_housing
california = fetch_california_housing()
```

Сперва следует ознакомиться со структурой тех данных, которые мы получили. Для этого выведем тип полученного объекта:

```
type(california)
```

```
sklearn.utils.Bunch
```

Это специальный тип данных библиотеки *sklearn*, который похож по своему устройству на обычный словарь. Поэтому посмотрим, какие ключи есть в этом словаре:

```
california.keys()
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
```

Особый интерес здесь представляют поля *data* и *target*, которые содержат именно исходные атрибуты и вектор значений целевой переменной. Выведем их тип:

```
print(type(california.data), type(california.target))
```

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

Так как это массивы *numpy*, то можно посмотреть их форму:

```
print(california.data.shape, california.target.shape)
```

```
(20640, 8) (20640,)
```

Получается, что в данных более 20 тысяч строк и 8 атрибутов. Дополнительно можно еще вывести описание датасета для получения дополнительной информации.

Теперь с данными можно работать разными способами. Для удобства анализа мы объединим все массивы в датафрейм:

```
data = pd.DataFrame(california.data, columns = california.feature_names)
data['Price'] = california.target
data.head()
```

index	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Lon
0	8.3252	41.0	6.984126984126984	1.0238095238095237	322.0	2.5555555555555554	37.88	-12
1	8.3014	21.0	6.238137082601054	0.9718804920913884	2401.0	2.109841827768014	37.86	-12
2	7.2574	52.0	8.288135593220339	1.073446327683616	496.0	2.8022598870056497	37.85	-12
3	5.6431	52.0	5.8173515981735155	1.0730593607305936	558.0	2.547945205479452	37.85	-12
4	3.8462	52.0	6.281853281853282	1.0810810810810811	565.0	2.1814671814671813	37.85	-12

Проверим данные на наличие пропущенных значений:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   MedInc      20640 non-null  float64
 1   HouseAge    20640 non-null  float64
 2   AveRooms    20640 non-null  float64
 3   AveBedrms   20640 non-null  float64
 4   Population  20640 non-null  float64
 5   AveOccup    20640 non-null  float64
 6   Latitude    20640 non-null  float64
 7   Longitude   20640 non-null  float64
 8   Price       20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

Видим, что пропусков в данных нет. Кроме того, видно, что все данные выражены в численных шкалах. Значит, особенной обработки данный датасет не требует, он уже достаточно чистый. Теперь можно вывести основную статистику по датасету:

```
data.describe().round(2)
```

index	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	Price
count	20640.0	20640.0	20640.0	20640.0	20640.0	20640.0	20640.0	20640.0	20640.0
mean	3.87	28.64	5.43	1.1	1425.48	3.07	35.63	-119.57	2.07
std	1.9	12.59	2.47	0.47	1132.46	10.39	2.14	2.0	1.15
min	0.5	1.0	0.85	0.33	3.0	0.69	32.54	-124.35	0.15
25%	2.56	18.0	4.44	1.01	787.0	2.43	33.93	-121.8	1.2
50%	3.53	29.0	5.23	1.05	1166.0	2.82	34.26	-118.49	1.8
75%	4.74	37.0	6.05	1.1	1725.0	3.28	37.71	-118.01	2.65
max	15.0	52.0	141.91	34.07	35682.0	1243.33	41.95	-114.31	5.0

Теперь выделим целевую переменную и факторы:

```
y = data['Price']
X = data.drop('Price', axis=1)
```

Выведем форму получившихся массивов:

```
y.shape, X.shape
```

```
((442,), (442, 10))
```

Приступим к обучению и оценке качества модели. Из набора линейных моделей библиотеки *sklearn* импортируем линейную регрессию:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
```

Как и в предыдущих работах выведем коэффициенты модели, так как в линейных моделях они имеют некоторый смысл:

```
print("Coefficients: \n", model.coef_)
```

```
Coefficients: 
 [ 4.36693293e-01  9.43577803e-03 -1.07322041e-01  6.45065694e-01
 -3.97638942e-06 -3.78654265e-03 -4.21314378e-01 -4.34513755e-01]
```

Выведем коэффициенты вместе с названиями соответствующих атрибутов:

```
_ = [print(k, v) for k, v in zip(X.columns, model.coef_)]
```

```
MedInc 0.4366932931343245
HouseAge 0.009435778033237972
AveRooms -0.10732204139090447
AveBedrms 0.645065693519812
Population -3.976389421211576e-06
AveOccup -0.003786542654971
Latitude -0.42131437752714385
Longitude -0.43451375467477743
```

Проанализируйте эти данные вместе с интерпретацией атрибутов датасетов и сделайте вывод о том, какие факторы как влияют на цену недвижимости в Калифорнии.

Как и в модели линейной регрессии, данный вектор не включает в себя свободный коэффициент. Он хранится в отдельном поле класса:

```
print("Intercept: \n", model.intercept_)
```

```
Intercept: 
 -36.94192020718441
```

Сделаем предсказания модели и выведем на экран первые несколько точек:

```
y_pred = model.predict(X)
print(y_pred[:5])
```

```
[4.13164983 3.97660644 3.67657094 3.2415985  2.41358744]
```

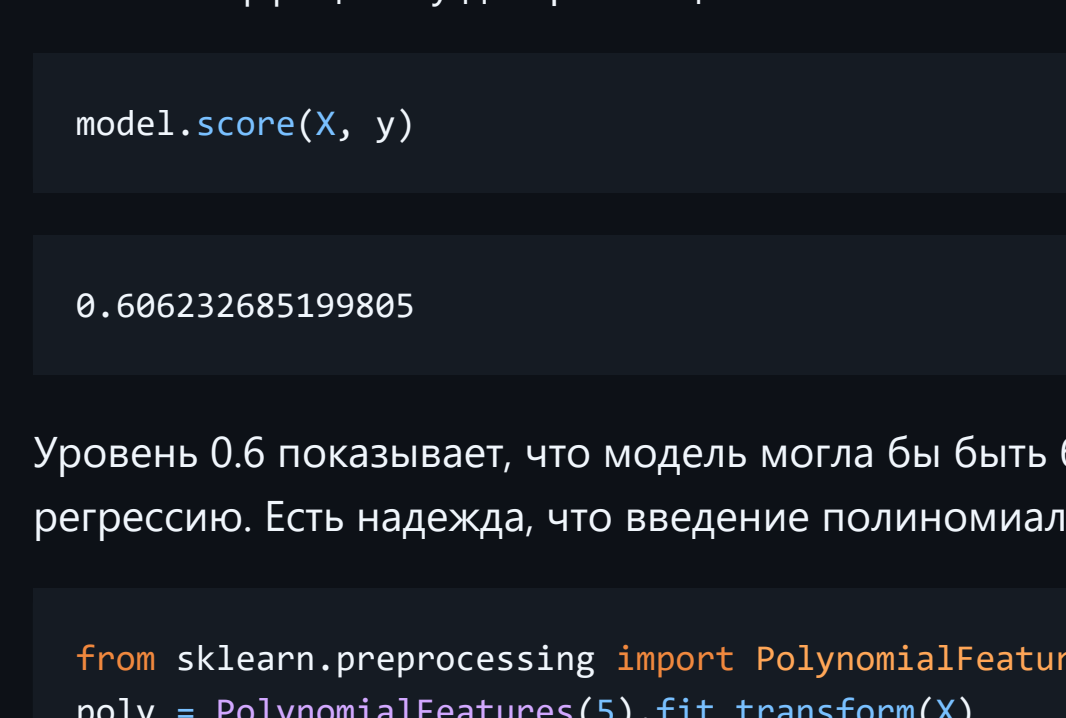
Для сравнения выведем реальные соответствующие значения целевой переменной:

```
print(y[:5])
```

```
0    4.526
1    3.585
2    3.521
3    3.413
4    3.422
Name: Price, dtype: float64
```

Конечно, так анализировать данные неудобно. Лучше построить график, демонстрирующий связь между реальными и предсказанными значениями целевой переменной:

```
plt.scatter(y_pred, y)
plt.plot(y, y, c='r')
```



На этом графике чем ближе точки к центральной линии, тем более точные прогнозы делает модель. В данном случае разброс довольно велик. Чтобы оценить эффективность модели численно, опять обратимся к встроенной метрике, реализованной методом *score* - коэффициенту детерминации:

```
model.score(X, y)
```

```
0.606232685199805
```

Уровень 0.6 показывает, что модель могла бы быть более точной. Давайте попробуем построить другую модель - полиномиальную регрессию. Есть надежда, что введение полиномиальных признаков может существенно увеличить точность модели:

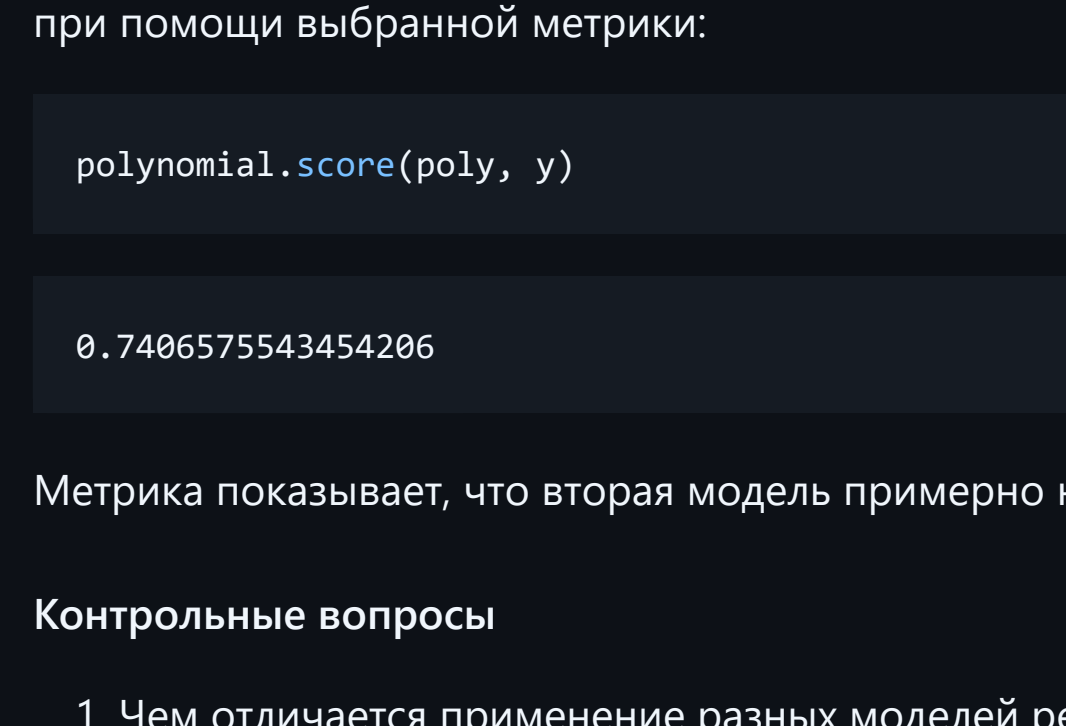
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(5).fit_transform(X)
```

В данном случае используем полиномиальные признаки пятой степени. Вы можете поэкспериментировать с другими степенями полинома. Построим предсказание для анализа:

```
polynomial = LinearRegression()
polynomial.fit(poly, y)
y_pred_poly = polynomial.predict(poly)
```

По этим данным можно построить график, подобный предыдущему:

```
plt.scatter(y_pred_poly, y)
plt.plot(y, y, c='r')
```



По этому графику можно сделать вывод, что модель стала несколько более точной. Но более конкретно это улучшение можно увидеть при помощи выбранной метрики:

```
polynomial.score(poly, y)
```

```
0.7406575543454206
```

Метрика показывает, что вторая модель примерно на 14 процентных пунктов лучше, чем первая.

Контрольные вопросы

1. Чем отличается применение разных моделей регрессии в бибилиотеке *sklearn* от моделей классификации?
2. Что показывает коэффициент детерминации для модели регрессии?
3. Какое значение имеют коэффициенты линейной регрессии?
4. Какие атрибуты имеет объект линейной регрессии?

Задания для самостоятельного выполнения

1. Какую еще информацию можно вывести для обученной модели? Попробуйте изменить аргументы при создании модели и посмотрите, как это влияет на качество предсказания.
2. Попробуйте применить к той же задаче другие модели регрессии. Для каждой из них выведите визуализацию регрессии и оценку точности. Рекомендуется исследовать следующие модели:
   i. Метод опорных векторов
      a. Без ядра
      b. С гауссовым ядром
      c. С полиномиальным ядром
   ii. Метод ближайших соседей
   iii. Многослойный перцептрон
   iv. Дерево решений
   v. (*) Другие методы:
      a. Гребневую регрессию
      b. Регрессию Лассо
      c. Регрессию ElasticNet
      d. Случайный лес
      e. Беггинг
      f. Другие модели по желанию
3. Напишите функцию, которая автоматически обучает все перечисленные модели и для каждой выдает оценку точности.
4. Повторите полностью анализ для другого набора данных - встроенного в *sklearn* датасета *diabetes*.